

A POLYNOMIAL APPROXIMATION SCHEME FOR A CONSTRAINED FLOW-SHOP SCHEDULING PROBLEM

LESLIE A. HALL

We present a polynomial approximation scheme for a two-machine flow shop scheduling problem with the additional constraint that each job has a release date, when it first becomes available for processing. This is one of the first such results for multiple-task scheduling problems; previously, the best approximation result for this problem was an algorithm guaranteed to deliver a schedule of length at most $5/3$ times the optimal length. Our $(1 + \epsilon)$ -approximation algorithm is intuitively appealing because it is based on Johnson's algorithm, an optimization algorithm for the problem without release dates.

1. Introduction. In the last twenty-five years, combinatorial scheduling theory has emerged as a major area of research. Deterministic scheduling problems are defined in terms of limited resources, called *machines*, and activities, or *jobs*, each consisting of one or more *tasks* that must be carried out on the machines. Each machine can process at most one task at a time. We distinguish two classes of scheduling problems: those with jobs requiring a single task, which can be performed by any one of the machines; and those whose jobs consist of multiple tasks, each of which must be performed on a specific machine or set of machines. The latter category is further partitioned into open-, flow-, and job-shop scheduling models (see Lawler, Lenstra, Rinnooy Kan, and Shmoys (1989) for a description of these models).

Most multiple-task scheduling problems are $\mathcal{NP}$ -hard. For such problems, it is natural to search for an *approximation algorithm*, an algorithm guaranteed to deliver a solution with value within a given factor of the optimal value. There is a recent but established tradition of studying approximation algorithms for scheduling problems; indeed, the first worst-case analysis of an approximation algorithm for *any* combinatorial optimization problem was carried out by Graham (1966), for a scheduling problem. Researchers have focused mainly on single-task models, however; little is known about optimization or approximation algorithms for multiple-task models.

One of the few results known for multiple-task models is Johnson's optimization algorithm for the two-machine flow shop problem (Johnson 1954). In a two-machine flow shop, each of n two-task jobs must be processed first on machine M_1 and then on machine M_2 . No preemption is allowed, and a job cannot begin processing on M_2 until it has completed processing on M_1 (although there can be time between when a job completes processing on M_1 and begins processing on M_2). The object is to minimize the time at which M_2 finishes processing all jobs. In the notation of Graham, Lawler, Lenstra, and Rinnooy Kan (1979), this model is denoted $F2||C_{\max}$.

Received December 5, 1990; revised August 27, 1992.

AMS 1980 subject classification. Primary: 90B35; Secondary: 68C25.

OR/MS Index 1978 subject classification. Primary: 589 Production/Scheduling/Flow-Shop. Secondary: 014 Analysis of Algorithms/Suboptimal.

Key words. Shop scheduling, approximation algorithms, flow shops, release dates, polynomial approximation schemes.

In this paper, we study a constrained version of the two-machine flow shop problem in which each job j has a *release date* r_j , when it first becomes available for processing on M_1 . We denote this problem $F2|r_j|C_{\max}$.

This problem is strongly $\mathcal{NP}$ -hard (Lenstra, Rinnooy Kan and Brucker 1977). Potts (1985) showed that several heuristics for $F2|r_j|C_{\max}$ produce schedules that are no more than twice as long as the optimal schedule; in other words, they are 2-approximation algorithms. He also constructed a 5/3-approximation algorithm, based on running one of the 2-approximation algorithms repeatedly on modified versions of the original instance. As far as we know, these are the only approximation results known for this problem, until now. In general, the best approximation result that we can hope to achieve is a *fully polynomial approximation scheme*, a family of algorithms $\{A_\epsilon\}$ with the property that, for any fixed $\epsilon > 0$, A_ϵ produces a schedule of length at most $(1 + \epsilon)$ times the length of the optimal schedule and has running time polynomial in n and $1/\epsilon$. However, because this scheduling problem is strongly $\mathcal{NP}$ -hard, there is no such scheme for it unless $\mathcal{P} = \mathcal{NP}$ (Garey and Johnson 1979). Consequently, the best result we can hope to obtain is a *polynomial approximation scheme*, namely, a family of $(1 + \epsilon)$ -approximation algorithms whose running times can depend in an arbitrary manner on $1/\epsilon$.

In this paper we construct a polynomial approximation scheme for $F2|r_j|C_{\max}$. It is based on the notion of an *outline scheme*, which Hall and Shmoys introduced for a class of single-task scheduling problems (Hall and Shmoys 1989, 1990, 1992). The following simple idea motivates this technique. For combinatorial optimization problems, often having some information about the optimal solution allows us to construct a near-optimal (or even optimal) solution in polynomial time. A strategy for solving the problem in general would then be to test every possibility for this information. Since at least one of the possibilities corresponds to an optimal solution, we are then guaranteed to find a near-optimal (or optimal) solution ourselves. Of course, the drawback to this strategy is that, in general, any set of information large enough to be useful has an exponential number of possible values. Our challenge is therefore to devise a set of information that is at once detailed enough to be useful but small enough that we can test every possible value for the information in polynomial time. We state these ideas more precisely in §2.

Our polynomial approximation scheme is one of the first results of its kind for multiple-task scheduling systems. In fact, we know of only one other strongly $\mathcal{NP}$ -hard, multiple-task problem for which a polynomial approximation scheme is known: Kovaliev (1985) claims a polynomial approximation scheme for the two-stage flow shop problem (without release-date constraints) in which each stage has a fixed number of machines available. The algorithm that we present is intuitively appealing for two reasons. First, it distinguishes jobs with long processing times from those with short processing times and thus reinforces our sense that long jobs are the ones that cause combinatorial headaches in scheduling. Second, it is closely based on Johnson's algorithm for $F2||C_{\max}$ (Johnson 1954). In some sense, despite the complications that release dates create, Johnson's algorithm remains fundamental to the structure of good solutions.

The rest of this paper is organized as follows. In §2 we present some preliminary concepts that allow us to simplify our view of the problem. In §3, we state the core of the $(1 + \epsilon)$ -approximation algorithm—namely, a subroutine that receives an *outline* as input, and delivers a schedule with length no more than $(1 + \epsilon)$ times the length of the shortest schedule with the input outline. We verify the performance guarantee of the subroutine by considering a series of steps that takes a schedule delivered by our subroutine and transforms it into the input schedule, without changing the length by too much. In this way, we show that the heuristic schedule cannot be much longer

than the input schedule. Section 4 contains two of the more complicated proofs of lemmas stated in §3. Finally, in §5 we construct the polynomial approximation scheme based on the subroutine and establish its running time.

2. Preliminaries. To begin our analysis of $F2|r_j|C_{\max}$, we observe that, for a two-machine flow shop, we can restrict our attention to *permutation schedules*, schedules for which the job processing order is the same on M_1 and M_2 (Johnson 1954). Indeed, any feasible schedule can be systematically transformed, via adjacent interchanges, into a permutation schedule that is at least as good as the original schedule, and in particular there is always a permutation schedule that is optimal.

Next we introduce notation. For a given instance of $F2|r_j|C_{\max}$, the processing time of job j on M_1 is a_j and on M_2 is b_j , $j = 1, \dots, n$, and we assume that both a_j and b_j are strictly greater than zero. (Normally, the flow shop model allows one of a_j or b_j to be zero; we require strict positivity in order to simplify the presentation. At the end of §3, we describe how our algorithm can easily be modified to handle the case of zero-length processing times.) Each job has a release date $r_j \geq 0$, when it becomes available for processing on M_1 . A schedule Σ for the instance is defined by the starting times of each job j on M_1 and M_2 , denoted σ_j and τ_j , respectively. Sometimes we will describe a schedule simply by a permutation of the jobs, referring to the *active schedule* that corresponds to that permutation, i.e., a schedule that, while respecting the given permutation, contains no unforced idle time. From now on, we will assume that all schedules are active permutation schedules; occasionally, we will need to consider permutation schedules that are not active, but we will explicitly specify this condition. We use $z(\Sigma)$ to denote the length of Σ , and z^* to denote the length of the optimal schedule. We assume that all given data are integral.

Next we formally introduce the notion of a $(1 + \epsilon)$ -approximate outline scheme (Hall 1989, Hall and Shmoys 1989).

DEFINITION. An *outline scheme* for a problem is a labeling of feasible solutions with the property that, for each feasible solution x , the associated label, called an *outline*, is polynomial in size and describes some characteristics of x .

In other words, for any instance of the problem, an outline scheme partitions the set of all feasible schedules into subsets corresponding to outlines, which characterize some aspect or aspects of the schedules. For example, one outline scheme for an instance of our problem labels a schedule Σ with the set of all starting times on M_1 in Σ , $\sigma_1, \dots, \sigma_n$. Of course, this outline is particularly useful because with it we can construct a schedule guaranteed to be as good as Σ by simply constructing the active permutation schedule corresponding to the permutation indicated by the σ_j . Most outlines reveal only partial information about their schedules; still, this information might be sufficient to allow us to construct, in polynomial time, a schedule that is nearly as good as any schedule corresponding to the outline.

DEFINITION. For $\epsilon > 0$, a $(1 + \epsilon)$ -approximate outline scheme for a minimization problem is an outline scheme and a polynomial-time algorithm A (requiring an outline ω as input) with the following property: given any instance I and an outline ω of I (which by definition corresponds to at least one feasible solution of I), Algorithm A delivers a feasible solution to I of value at most $(1 + \epsilon)$ times the value of any feasible solution of I that is labeled with ω .

In particular, given an outline corresponding to an optimal solution to the instance, the algorithm delivers a solution of value at most $(1 + \epsilon)z^*$. Moreover, if for every instance the number of distinct outlines associated with a $(1 + \epsilon)$ -approximate outline scheme is polynomial in the size of the instance, then by running the algorithm on every possible outline for the instance we obtain a $(1 + \epsilon)$ -approximation algorithm

for the problem. We refer to a 1-approximate outline scheme as an optimal outline scheme.

If all jobs are available at time $t = 0$, the problem can be solved by Johnson's algorithm (Johnson 1954):

Partition the jobs into two sets, those j with $a_j \leq b_j$, and those with $a_j > b_j$. Schedule the former set of jobs first, in order of nondecreasing a_j ; then schedule the remaining jobs, in order of nonincreasing b_j .

Next we introduce a lemma that allows us to focus on a special, restricted version of the problem; it is entirely analogous to a lemma for the single-machine problem $1|r_j|L_{\max}$ in Hall and Shmoys (1992).

LEMMA 1. *Given a polynomial approximation scheme for the restricted version of $F2|r_j|C_{\max}$ in which there is a fixed (but arbitrary) number of distinct release dates, then there is a polynomial approximation scheme for $F2|r_j|C_{\max}$.*

PROOF. Consider an instance of $F2|r_j|C_{\max}$, and let $\phi = \max_j r_j$. Clearly $\phi \leq z^*$, since some job begins processing at or after time $t = \phi$. Consider the following algorithm for the unrestricted version:

Round each release date r_j down to the nearest multiple of $\phi\epsilon/2$. Apply a $(1 + \epsilon/2)$ -approximation algorithm that can handle $1 + 2/\epsilon$ distinct release dates. Add $\phi\epsilon/2$ to each job's starting time in the schedule just created.

Let $\bar{z}$ and z^* be the optimal values of the rounded and original instances, respectively. Since the rounded problem is less constrained than the original, clearly $\bar{z} \leq z^*$. The final schedule delivered is clearly feasible for the original problem, and its length is bounded by $(1 + \epsilon/2)\bar{z} + \phi\epsilon/2 \leq (1 + \epsilon)z^*$. $\square$

Throughout this paper, we focus on an instance with a fixed but arbitrary number κ of distinct release dates, which we denote by $\rho_1 < \rho_2 < \dots < \rho_\kappa$. For notational convenience we set $\rho_{\kappa+1} = \infty$.

DEFINITION. For a given schedule Σ , suppose that $\rho_i \leq \sigma_j < \rho_{i+1}$. Then we call ρ_i the *effective release date* (with respect to Σ) of job j , and we denote it by $\bar{r}_j$.

Note that $r_j \leq \bar{r}_j$. The effective release date of a job is the largest of the κ release dates that a job could have without destroying the feasibility of Σ . Because for each i the jobs j with $\bar{r}_j = \rho_i$ are scheduled consecutively in Σ , we can view the schedule as being partitioned into blocks of jobs corresponding to the sets $\{j: \bar{r}_j = \rho_i\}$, $i = 1, \dots, \kappa$. The following lemma states that, if the jobs in each of the blocks are reordered according to Johnson's algorithm, the resulting schedule is at least as good as Σ .

LEMMA 2. *Given the effective release dates for a schedule Σ , let π_i be the permutation of the set $\{j: \bar{r}_j = \rho_i\}$ given by Johnson's algorithm, for $i = 1, \dots, \kappa$. Then the schedule corresponding to the permutation $\pi_1, \dots, \pi_\kappa$ has length at most $z(\Sigma)$.*

PROOF. Let $\Psi_i = \{j: \bar{r}_j = \rho_i\}$, $i = 1, \dots, \kappa$. First, consider the schedule Σ . Notice that each block Ψ_i begins processing on M_1 at or after its associated release date. Furthermore, the only idle periods on M_1 occur between two blocks, and in that case the later of the two blocks begins processing precisely at its associated release date. Next, let π_i denote the permutation of Ψ_i given by applying Johnson's algorithm to the set Ψ_i , $i = 1, \dots, \kappa$, and let Σ_j denote the active permutation schedule corresponding to the ordering $\pi_1, \dots, \pi_\kappa$. In Σ_j , it is possible that, for some i , a job of Ψ_i might begin processing before time ρ_i . Let Σ' denote the schedule given by $\pi_1, \dots, \pi_\kappa$, with the additional restriction that no job of Ψ_i begins processing before time ρ_i , for all i . Notice that Σ' might not be an active schedule; however, except with respect to the additional restriction, Σ' is an active schedule. Clearly $z(\Sigma_j) \leq z(\Sigma')$. To prove the lemma we will show that $z(\Sigma') \leq z(\Sigma)$.

We will show by induction that the set Ψ_i completes processing on M_2 at least as early in Σ' as in Σ , for all i . We note that the inductive claim for $i = \kappa$ yields $z(\Sigma') \leq z(\Sigma)$.

Observe that, for each i , the set Ψ_i completes processing on M_1 at precisely the same time in Σ' and Σ . We now turn to the inductive claim. For $i = 1$, the hypothesis is equivalent to Johnson's algorithm being optimal for the problem without release dates because all jobs of Ψ_1 and both machines M_1 and M_2 are available at the same time ρ_1 . Thus Johnson's algorithm is guaranteed to attain the earliest possible finishing time on M_2 for this set.

Now we assume that the claim is true for Ψ_i , and we consider Ψ_{i+1} . The hypothesis essentially claims that using Johnson's algorithm to order Ψ_{i+1} is optimal even if M_2 becomes available for processing Ψ_{i+1} after M_1 does. By introducing a dummy job that compensates for this time difference, we can reduce the inductive step to the case $i = 1$.

Suppose that, in Σ' , Ψ_i completes processing on M_1 and M_2 at times $\bar{\sigma}$ and $\bar{\tau}$, respectively (which are at least as early as the times this set completes in Σ , by the observation made above and the inductive hypothesis). Then M_1 and M_2 are available to process the jobs of Ψ_{i+1} at times

$$\sigma := \max\{\rho_{i+1}, \bar{\sigma}\} \quad \text{and} \quad \tau := \max\{\rho_{i+1}, \bar{\tau}\},$$

respectively. We introduce a dummy job 0 that has $a_0 = 0$ and $b_0 = \tau - \sigma$, and we use Johnson's algorithm to schedule the set $\Psi_{i+1} \cup \{0\}$, beginning at time σ . Since $a_0 = 0$, job 0 can be scheduled first in Johnson's algorithm, and it completes processing on M_2 at time τ . The placement of the dummy job keeps the remaining jobs from overlapping on M_2 with the previously scheduled set, Ψ_i . The remaining jobs get scheduled according to Johnson's algorithm, which generates the earliest possible finishing time for Ψ_{i+1} on M_2 , not only with respect to the "dummy" problem, but with respect to the original problem as well. Thus the inductive step is proved. $\square$

The lemma shows that once we know the effective release date of every job for a schedule, we can construct a schedule guaranteed to be at least as good as the original schedule.

COROLLARY 1. *A labeling consisting of the set of effective release dates for a schedule is the basis of an optimal outline scheme.*

3. A $(1 + \epsilon)$ -approximate outline scheme. Of course, the optimal outline scheme implied by Corollary 1 cannot be used as the basis of a polynomial approximation scheme, because there are on the order of κ^n different possible ways of assigning effective release dates to the jobs. But as we shall see, we will be able to modify this scheme so that we lose only a slight amount of accuracy in our derived solutions while requiring only a fixed number (in terms of ϵ) of different possible outlines to try.

In particular, rather than having an outline specify the effective release dates of all of the jobs, we will specify only the effective release dates of the "largest" jobs. In addition, we will need some information about where the so-called "small" jobs should be scheduled, but the outline will not give specific information about each small job. Rather, this information will be aggregated, so that there is only a fixed number of possible choices for the information. Based on the aggregate information, we will schedule the small jobs using a greedy rule.

DEFINITION. A job j is a *large job* if its processing time on M_1 is large with respect to the total processing time on M_1 ,

$$a_j \geq \frac{\epsilon}{3\kappa} \sum_{j=1}^n a_j.$$

In particular, for any schedule Σ and each small job j , $a_j < (\epsilon/3\kappa)z(\Sigma)$ since $\sum_{j=1}^n a_j \leq z(\Sigma)$. We denote the sets of large and small jobs as A and B , respectively.

Let us fix a permutation schedule Σ . Let $\delta = (\epsilon/3\kappa)\sum_{j=1}^n a_j$, so that large jobs are those j with $a_j \geq \delta$. For $i = 1, \dots, \kappa$, let

$$I_i := \{j: \bar{r}_j = \rho_i, j \in A\}, \quad \text{and} \quad H_i := \{j: \bar{r}_j = \rho_i, j \in B\},$$

and let

$$L_i := \sum_{j \in H_i} a_j.$$

Let $\nu_i := \lfloor L_i/\delta \rfloor$, and $N_i = \nu_i \delta$, the approximate amount of small-job-time (rounded up to the nearest multiple of δ) that is spent on M_1 processing small jobs j with $\bar{r}_j = \rho_i$.

THEOREM 1. *The information (I, ν) ,*

$$I = (I_1, \dots, I_\kappa) \quad \text{and} \quad \nu = (\nu_1, \dots, \nu_\kappa),$$

with Algorithm 1 below, is a $(1 + \epsilon)$ -approximate outline scheme for $F2|r_j|C_{\max}$.

The rest of this section is devoted to proving this theorem. In particular, we need to show that, for any schedule Σ with outline (I, ν) , upon input (I, ν) Algorithm 1 delivers a schedule Σ' with $z(\Sigma') \leq (1 + \epsilon)z(\Sigma)$. Looking ahead to the polynomial approximation scheme, we note that it is not hard to see that there is only a fixed number of possibilities for the vector (I, ν) (see Theorem 2 in §4).

Recall that Lemma 2 implies that a schedule Σ is essentially characterized by its effective release dates. The effective release dates of Σ induce a partition $I_1 \cup H_1, I_2 \cup H_2, \dots, I_\kappa \cup H_\kappa$; for any such partition, we shall refer to the set $I_i \cup H_i$ as the *i th release interval set* of jobs, for $i = 1, \dots, \kappa$. By Lemma 2, if we knew this partition for Σ , we could construct a schedule guaranteed to be as good as Σ . Although the outline specifies the sets I_i , however, instead of the sets H_i it specifies only numbers, N_i . From this information the algorithm will partition the set of small jobs as $J_1, J_2, \dots, J_\kappa$, and will then use Johnson's algorithm on the overall partition $I_1 \cup J_1, \dots, I_\kappa \cup J_\kappa$, as in Lemma 2.

In order to select the sets J_i , the algorithm uses a simple greedy rule that Lemma 3 will motivate. Each set J_i will be selected so that its M_1 -processing time is approximately equal to N_i , if possible; and its M_2 -processing time is as large as possible, subject to the first constraint. Thus "available" small jobs will be chosen in order of nonincreasing b_j/a_j . We now state the core of the algorithm, which receives an outline (I, ν) of a schedule Σ as part of its input. Recall that $N_i = \nu_i \delta$.

Algorithm 1

$J_i := \emptyset, i = 1, \dots, \kappa;$

for $i = 1, \dots, \kappa$,

 let $W_i := \{\text{small jobs } j: j \notin J_k, \text{ for any } k < i, \text{ and } r_j \leq \rho_i\};$

 let $\{j_1, \dots, j_m\}$ denote the jobs in W_i , numbered so that $b_{j_1}/a_{j_1} \geq \dots \geq b_{j_m}/a_{j_m};$

if $\sum_{j \in W_i} a_j < v_i \delta$ then set $J_i = W_i$;
 else let J_i be the minimal set of the form $\{j_1, \dots, j_l\}$ with $\sum_{j \in J_i} a_j \geq v_i \delta$;
 let π_i be the permutation of $I_i \cup J_i$ given by Johnson's algorithm applied to
 the set $I_i \cup J_i$, disregarding release dates;
 schedule all jobs in the order given by $\pi_1, \pi_2, \dots, \pi_k$.

Next, we provide intuition to motivate the greedy manner in which the small jobs are selected in Algorithm 1.

LEMMA 3. Consider two instances I and $\tilde{I}$ of $F2|r_j|C_{\max}$ with identical jobs except for two jobs j and k , with processing times $\tilde{a}_k = a_k$, $\tilde{a}_j = a_j$, $\tilde{b}_k = b_k + \alpha$, and $\tilde{b}_j = b_j - \alpha$, for some α with $b_j \geq \alpha > 0$. For a given permutation with k before j , let Σ and $\tilde{\Sigma}$ be the corresponding schedules for I and $\tilde{I}$, respectively. Then $z(\tilde{\Sigma}) \leq z(\Sigma)$.

PROOF. It is sufficient to show that job j completes processing on M_2 in $\tilde{\Sigma}$ at least as early as in Σ . Let τ_j and $\tilde{\tau}_j$ be the starting time of job j on M_2 in Σ and $\tilde{\Sigma}$, respectively, so that $t = \tau_j + b_j$ and $\tilde{t} = \tilde{\tau}_j + \tilde{b}_j$ are the finishing times of job j on M_2 in Σ and $\tilde{\Sigma}$, respectively. Observe that $b_k = b_k + \alpha$ implies $\tilde{\tau}_j \leq \tau_j + \alpha$, because α is the most that job k could have delayed j 's starting time on M_2 in $\tilde{\Sigma}$ as compared to Σ . Thus

$$\tilde{t} = \tilde{\tau}_j + \tilde{b}_j \leq (\tau_j + \alpha) + (b_j - \alpha) = t. \quad \square$$

We can interpret Lemma 3 as follows. For a given schedule, if we reassign a chunk of M_2 -processing time from one job to an earlier-scheduled job, then the resulting schedule cannot be longer than the original one, and it might even be shorter, because some idle time on M_2 might get eliminated. The two-job example in Figure 1 illustrates this effect: let $r_1 = r_2 = 0$, $a_1 = b_1 = b_2 = 1$, and $a_2 = 2$, and suppose that we schedule job 1 first. Then by shifting one unit of time from b_2 to b_1 , the overall schedule becomes shorter. We note that, for two jobs with the same amount of M_1 -processing time, the lemma implies that the job with larger M_2 -processing time should get priority over the other. This priority rule is a special case of the one used in Algorithm 1 (which gives priority to jobs j with large b_j/a_j).

We refer to the heuristic schedule generated by Algorithm 1 as Σ' . To show that the algorithm does indeed deliver a schedule within a factor of $(1 + \epsilon)$ of $z(\Sigma)$, we will introduce a series of transformations that essentially changes the heuristic schedule Σ' into the schedule Σ . We will then show that each step in the transformation cannot shorten the resulting schedule by too much, so that overall $z(\Sigma') < (1 + \epsilon)z(\Sigma)$. Throughout the process, jobs may be reassigned to new release interval sets; with any assignment of jobs to release interval sets, we associate a corresponding schedule, obtained by scheduling each interval set by Johnson's algorithm and concatenating the interval-set schedules. Furthermore, we assume that this schedule has the structure introduced in Lemma 2, namely that each release interval set begins processing at or after its release date. The transformation is simply a proof technique used to show that the schedule generated by Algorithm 1 is not very different from Σ .

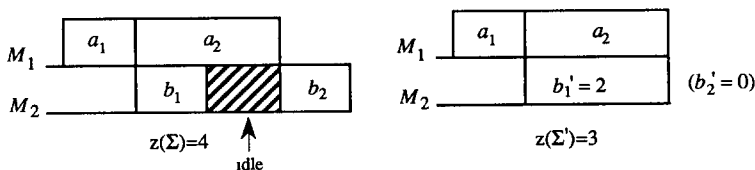


FIGURE 1. An example illustrating how moving M_2 -processing time back in the schedule shortens it.

Let us explain these transformations qualitatively. We begin with the heuristic schedule's assignment, which assigns the large jobs to the same release interval sets as Σ does, and which assigns approximately the correct amount of M_1 small-job-time to each release interval set. In Step 2, we break the small jobs up into smaller jobs so that all small jobs have exactly the same M_1 -processing requirement of one unit. In Step 3, we reassign some of these unit-length jobs to later release interval sets, so that each interval set now has exactly the same amount of small-job-time on machine M_1 as Σ does. (We reassign the jobs in a particular iterative, greedy manner, so as not to destroy a certain dominance relationship between the M_2 small-job-time for the current schedule and that for Σ . In essence, for any $k = 1, \dots, \kappa$, the total M_2 small-job-time assigned to the first k interval sets in the current schedule will be at least as large as that in the first k interval sets of Σ .) We will need to prove that, by the time we consider interval set i , the number of unit-length jobs assigned to interval set i is at least L_i . In Step 4, we reassign chunks of M_2 -time so that each interval set ends up with exactly the same amount of M_2 small-job-time as Σ has; in particular, the new sets J'_i are isomorphic to the sets H'_i (defined below). Again, this process is carried out iteratively, to ensure that the transformation only requires moving M_2 -time to later intervals; in that way, Lemma 3 will ensure that this step cannot shorten the schedule. Finally, in Step 5 the unit-length jobs are reassembled into jobs isomorphic to the original small jobs, now with interval assignments that correspond to the schedule Σ . This assembly is possible because of the transformations in Steps 2 to 4. Recall that all job lengths are assumed to be integral.

Transforming schedule Σ' to schedule Σ .

(1) Begin with the schedule generated by Algorithm 1, based on the sets $I_1, \dots, I_\kappa$ and $J_1, \dots, J_\kappa$.

(2) Replace each small job $j \in B$, with processing times a_j and b_j , by a_j jobs, each having processing times 1 and b_j/a_j , respectively, on M_1 and M_2 . (We refer to these new jobs as *unit jobs*, because their processing time on M_1 is one unit, and we call the set of all unit jobs B' .) Each set of small jobs J_i (or H_i) from the old instance induces a set of unit jobs in the new instance that we refer to as J'_i (or H'_i). We refer to the M_2 -processing time of the new unit jobs j as b'_j , and also as c'_j . The values b'_j will be modified in Step 4, while the values c'_j will remain fixed throughout. (The b'_j and c'_j are allowed to be noninteger.)

(3) For $i := 1, \dots, \kappa - 1$,

Let $\mu = |J'_i| - L_i$. (Comment. $\mu \geq 0$, as we show in Lemma 5.)

Remove from J'_i the μ unit jobs with smallest b'_j and reassign them to J'_{i+1} .

(After this step $|J'_i| = |H'_i|$, for all i .)

(4) For $i := 1, \dots, \kappa - 1$

(i is the index of the set J'_i from which M_2 -time is about to be removed)

Let $m = |J'_i| = |H'_i|$ and temporarily renumber the jobs in J'_i so that

$J'_i = \{1, 2, \dots, m\}$ and $b'_1 \geq \dots \geq b'_m$.

Let $H'_i = \{h_1, \dots, h_m\}$ so that $c'_{h_1} \geq \dots \geq c'_{h_m}$.

(We associate job $l \in J'_i$ with $h_l \in H'_i$.)

(*) For $j := 1, \dots, m - 1$

(j is the job of J'_i from which M_2 -time is about to be removed)

Let $\alpha = b'_j - c'_{h_j}$.

(Note that α is the amount of time that needs to be removed from b'_j to make it equal to c'_{h_j} ; and $b'_j \geq c'_{h_j}$, as we show in Lemma 7.)

Let $k = i + 1$.

(Index k represents the interval set to which M_2 -time is currently being moved.)

While $\alpha \neq 0$ repeat

(move M_2 -processing time to jobs in later release interval sets)
temporarily renumber the jobs in J'_k so that $J'_k = \{m+1, \dots, q\}$
and $b'_{m+1} \geq \dots \geq b'_q$;
let $H'_k = \{h_{m+1}, \dots, h_q\}$ such that $c'_{h_{m+1}} \geq \dots \geq c'_{h_q}$;
while there is a job $l \in J'_k$ with $b'_l < c'_{h_l}$
let $\eta = \min\{c'_{h_l} - b'_l, \alpha\}$
(η is the largest amount of time that can be moved from b'_j to
 b'_l without making b'_j too small or b'_l too large), and
set $b'_l := b'_l + \eta$;
set $b'_j := b'_j - \eta$;
set $\alpha = \alpha - \eta$;
set $k = k + 1$.

(5) Sets J'_i and H'_i are now isomorphic. Glue the unit-length jobs back together to get the original sets H_i . The current assignment (I_i, H_i) , $i = 1, \dots, \kappa$, is identical to that for Σ .

Steps 2 and 3 are the two steps in which the schedule might become shorter. We will show that the total amount of time by which the transformations shorten the schedule is bounded by $\epsilon z(\Sigma)$. We also need to prove that the steps described are indeed all possible, so that by the time we reach Step 5 we really do end up with the interval-set assignment that corresponds to schedule Σ .

Let the values of the schedules corresponding to Steps 1 through 5 be $z_1, \dots, z_5$, respectively. Note that $z_1 = z(\Sigma')$, and by Lemma 2, $z_5 \leq z(\Sigma)$.

LEMMA 4. *In Step 2, replacing the jobs in J_i by the unit jobs comprising J'_i shortens the schedule by less than δ , and thus*

$$z_2 > z_1 - \kappa\delta = z_1 - \epsilon P/3,$$

for $P = \sum_{j=1}^n a_j$.

PROOF. We denote the starting time of any job j before Step 2 as σ_j and of any job j after Step 2 as $\tilde{\sigma}_j$. To prove this lemma, we must understand something of the structure of the ordering generated by Johnson's algorithm on the set $J_i \cup I_i$. The subset of jobs from J_i with $a_j \leq b_j$ lie at the extreme early end of the ordering, before replacing those jobs with unit jobs; and after the replacement, an optimal ordering can be obtained by allowing those new jobs of J'_i corresponding to old jobs with $a_j \leq b_j$ to remain in the same order as the old jobs were, before the replacement. As for the jobs of J_i with $a_j > b_j$, the new jobs derived from the old drift to later positions in the schedule, since their M_2 -processing times become smaller. In particular, for some Johnson ordering of the jobs after the switch, the following statement holds: for any $k \in I_i$ and $j \in J_i$ with $a_j > b_j$, if $\sigma_j > \sigma_k$ and j' is derived from j , then $\tilde{\sigma}_{j'} > \tilde{\sigma}_k$.

Fix some value i , and assume that, for $l \leq i-1$, J'_l has already replaced J_l . Given that the schedule has already been adjusted for the first $i-1$ intervals, we wish to determine how much the schedule can be shortened by replacing J_i with J'_i . Let t_a and t_b be the starting time on M_1 and M_2 , respectively, of the job scheduled earliest from $I_i \cup J_i$ (before the replacement). Define job k to be the latest-scheduled job from $I_i \cup J_i$ with the property that M_2 is idle at some point during the processing of job k on M_1 . If no such idle period exists, then replacing J_i with J'_i will not shorten the schedule, since in this case the finishing time on M_2 for the entire set of jobs will

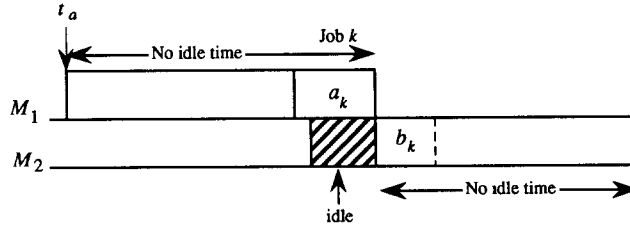


FIGURE 2. Changing the small jobs into unit jobs does not shorten the schedule too much.

remain

$$t_b + \sum_{j \in I_i} b_j + \sum_{j \in J_i} b_j = t_b + \sum_{j \in I_i} b_j + \sum_{j \in J'_i} b'_j.$$

We now delineate several cases for the type of job k .

Case 1. Job $k \in I_i$, $b_k \geq \delta$. If $k \in I_i$ and $b_k \geq \delta$, then the replacement will not shorten the schedule. To see this fact, we note that all of the jobs j of J_i with $a_j \leq b_j$ satisfy $\sigma_j < \sigma_k$, and those with $a_j > b_j$ must satisfy $\sigma_j > \sigma_k$, by the order in which Johnson's algorithm schedules jobs. But by the remarks made at the beginning of this proof, we know that for some Johnson ordering, all of the jobs scheduled before k remain scheduled before k (or their derived jobs do), and all of the jobs scheduled after remain after. Thus the finishing time of this part of the schedule both before and after is

$$t_a + \sum (a_j : \sigma_j \leq \sigma_k) + \sum (b_j : \sigma_j \geq \sigma_k)$$

(see Figure 2).

Case 2. Job $k \in J_i$, $a_k \leq b_k$. In this case, as in the last, the jobs originally scheduled before k remain before k , in some Johnson schedule, and those originally scheduled after remain after. In this case, the length of the schedule after replacing jobs in J_i is at most $a_k - 1$ units shorter than before (see Figure 3), and since $a_k < \delta$, the new schedule is less than δ units shorter than the old one.

Case 3. $a_k > b_k$, and (i) job $k \in J_i$, or (ii) $k \in I_i$ and $b_k < \delta$. In this case, the old schedule finishes $J_i \cup I_i$ at time

$$t_a + \sum (a_j : \sigma_j \leq \sigma_k) + \sum (b_j : \sigma_j \geq \sigma_k).$$

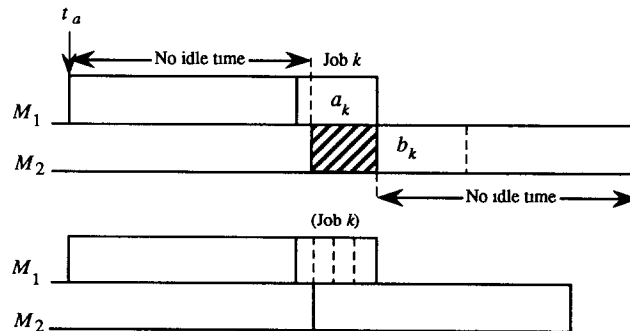


FIGURE 3. Changing the small jobs into unit jobs can shorten the schedule by at most the length of job k 's processing time on M_1 , minus one. This is because the two intervals marked "no idle time" will not overlap, after the replacement; thus the sum of the lengths of these intervals is a lower bound on the total length.

But the finishing time after the replacement is certainly bounded below by

$$t_a + \sum_{j \in J_i \cup I_i} a_j.$$

Since $a_k > b_k$, for all j with $\sigma_j > \sigma_k$ we have $a_j > b_j$. Thus

$$\sum (a_j: \sigma_j > \sigma_k) \geq \sum (b_j: \sigma_j > \sigma_k),$$

and the new finishing time is bounded below by

$$\begin{aligned} t_a + \sum_{j \in J_i \cup I_i} a_j &\geq t_a + \sum (a_j: \sigma_j \leq \sigma_k) + \sum (b_j: \sigma_j \geq \sigma_k) - b_k \\ &> t_a + \sum (a_j: \sigma_j \leq \sigma_k) + \sum (b_j: \sigma_j \geq \sigma_k) - \delta, \end{aligned}$$

which is less than δ units earlier than the old finishing time.

Thus by replacing J_i by J'_i the schedule is shortened at most δ units, and the schedule at the end of Step 2 is less than $\kappa\delta$ units shorter than the schedule before Step 2, i.e., $z_2 > z_1 - \kappa\delta$. $\square$

LEMMA 5. *When Step 3 begins iteration i , $|J'_i| - L_i \geq 0$; and thus after Step 3, $|J'_i| = L_i$.*

PROOF. Let us fix i . Before Step 3 begins, either $|J'_i| \geq L_i$, or else when J_i was constructed in Algorithm 1, it consisted of all remaining available small jobs. In the first case we are done. In the second case, by the time that iteration i of Step 3 is about to take place, we have $|J'_l| \leq L_l$, for all $l \leq i - 1$. Let $C := \{\text{small jobs } j: r_j \leq \rho_i\}$. Since J_i contains all remaining (available) small jobs, we must have

$$\sum_{l=1}^i |J'_l| = \sum_{l=1}^i \sum_{j \in J_l} a_j = \sum_{j \in C} a_j \geq \sum_{l=1}^i L_l.$$

Since $|J'_l| \leq L_l$ for $l = 1, \dots, i - 1$, we must have $|J'_i| \geq L_i$. In particular, after Step 3, $|J'_i| = L_i$ for all i . $\square$

LEMMA 6. *Step 3 shortens the schedule by less than $2\kappa\delta = 2\epsilon P/3$, and thus*

$$z_3 > z_2 - 2\kappa\delta = z_2 - 2\epsilon P/3,$$

for $P = \sum_{j=1}^n a_j$.

PROOF. Consider a point in Step 3 at which a unit job j is about to be reassigned to a later release interval set, and consider the schedule corresponding to the assignment just prior to the reassignment. Let job k be the last job in this schedule for which M_2 is idle during k 's processing on M_1 , and let X be the set of jobs up to and including k that form a nonidle block on M_1 . The length of the current schedule is determined by the M_1 starting time of the nonidle block X , plus the M_1 processing time on X , plus the M_2 processing time of job k and all subsequent jobs (see Figure 4). Therefore, the only way that the schedule might become shorter due to reassigning job j is if j originally belonged to the set X and ended up getting scheduled after all of the jobs in X . In that case, for that single reassignment the schedule could shrink by at most the processing time of job j on M_1 minus its processing time on M_2 , which is bounded by 1, since j is a unit job. Since the total small job M_1 -time assigned by Algorithm 1 to interval set i is less than $N_i + \delta < 2\delta + L_i$, the total

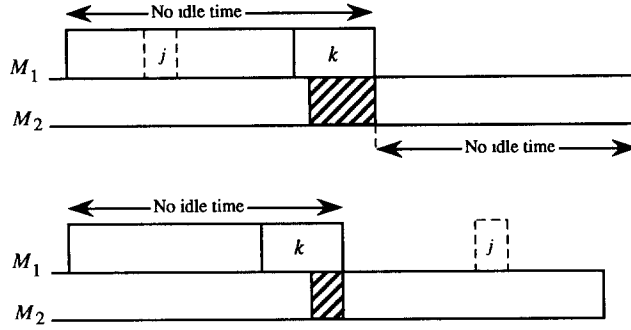


FIGURE 4. Reassigning unit job j in Step 3 can shorten the schedule by at most one unit. In the figure, the M_2 processing time of j is assumed to be minuscule.

number of unit jobs that get reassigned in this manner is bounded by $2\kappa\delta$, i.e.,

$$\sum_{i=1}^{\kappa-1} (N_i + \delta) - \sum_{i=1}^{\kappa-1} L_i,$$

and the result follows. $\square$

Lemmas 7 and 8 prove the correctness of Step 4 of the transformation; their proofs are in the next section. Central to the proofs is the greedy manner in which the small jobs were selected in Algorithm 1. In Step 4, M_2 -time is reassigned from earlier jobs to later jobs. Implicit in the transformation is the assumption that, whenever a particular job is considered as job j in Step 4, $b'_j \geq c'_{h_j}$. We need to prove that this is indeed the case.

LEMMA 7. *In Step 4, by the time that job j is compared with its corresponding job h_j at step (*), $b'_j \geq c'_{h_j}$.*

This fact—coupled with the fact from Lemma 8 that for each job j , α eventually reaches zero—implies that at the end of Step 4, each of the sets J'_i is isomorphic to H'_i , i.e., $b'_j = c'_{h_j}$ for all j . It is then routine, in Step 5, to recover the sets H_i corresponding to Σ .

LEMMA 8. *In Step 4, at the “While $\alpha \geq 0$ ” loop, α eventually reaches zero; and thus after any job j is processed at (*), $b'_j = c'_{h_j}$.*

LEMMA 9. *Steps 4 and 5 can only lengthen, not shorten, the transformed schedule, and thus $z_5 \geq z_4 \geq z_3$.*

PROOF. The only changes to the instance at Step 4 are to reassign a chunk of M_2 -time from one interval set to a later one; by Lemma 3, such transformations can only lengthen the resulting schedule. The problem instance before Step 5 is a relaxation of the problem instance after Step 5, in the sense that any schedule produced for the instance after Step 5 could be exactly imitated by the instance before Step 5 by simply placing the jobs to be glued together next to each other and delaying their M_2 -processing until all of the jobs in a group have been processed on M_1 . $\square$

We are now in a position to prove Theorem 1. Lemmas 4 through 9 imply that the schedule Σ' is bounded in length by

$$\begin{aligned} z(\Sigma') &= z_1 < z_2 + \kappa\delta < z_3 + 3\kappa\delta \\ &\leq z_5 + 3\kappa\delta \leq z(\Sigma) + 3\kappa\delta \\ &= z(\Sigma) + \epsilon P \leq (1 + \epsilon)z(\Sigma), \end{aligned}$$

since $P = \sum_{j=1}^n a_j$ is a lower bound on $z(\Sigma)$. $\square$

We conclude this section with a remark on zero-length processing times. The results of this section can easily be generalized to allow $a_j = 0$ or $b_j = 0$ for any job j . In Algorithm 1, if $a_k = 0$ for some job k , then k would be classified as a small job and it would receive priority above all small jobs with $a_j > 0$, which would still be ranked by nonincreasing b_j/a_j . Allowing $b_j = 0$ for some jobs has no effect on Algorithm 1. Finally, it is straightforward, albeit messy, to modify the transformation steps and lemmas that prove the correctness of Algorithm 1.

4. Proofs of Lemmas 7 and 8. In what follows, we work mainly with the original M_2 -values for the unit jobs, which are denoted as c'_j (see Step 2). The proof of Lemma 7 will be based on an induction argument given in the following lemma. To simplify the presentation, throughout the proofs of Lemmas 10 and 7 we assume that no two unit jobs have the same M_2 -processing time, c'_j . At first glance, this assumption might seem strange, since we know that, because of the manner in which the c'_j were constructed, many of them are identical. However, the assumption is not restrictive because we use it only to distinguish a linear order on these values. Such an order could be obtained without the assumption, for example by breaking ties based on an additional index.

In addition, we make the following observation, based on the greedy selection used to push unit jobs back in Step 3.

Observation. The unit-job assignments after Step 3 are identical to those that would have resulted if we had applied Algorithm 1 to an instance in which all of the small jobs had already been converted to unit jobs, and in which $N_i = L_i$, for all i .

Therefore, in the following discussion, we will generally view Algorithm 1 as acting directly on this modified instance.

LEMMA 10. *Let $1 \leq i \leq \kappa$, and let*

$$J(i) = \bigcup_{l=1}^i J'_l \quad \text{and} \quad H(i) = \bigcup_{l=1}^i H'_l,$$

with sets J'_l as they are defined after Step 3 has been carried out. Assume that the jobs are numbered so that $J(i) = \{1, \dots, m\}$, $c'_1 \geq \dots \geq c'_m$, $H(i) = \{h_1, \dots, h_m\}$, $c'_{h_1} \geq \dots \geq c'_{h_m}$. Then for $j = 1, \dots, m$, $c'_j \geq c'_{h_j}$.

PROOF. The proof is by induction on i . First we consider the case $i = 1$; note that $J(1) = J'_1$, $H(1) = H'_1$. Because of the greedy choice of small jobs in Algorithm 1 (and in Step 3), clearly $c'_j \geq c'_{h_j}$, at the start of Step 4, for all $j \in J'_1$. Thus the inductive hypothesis holds for $i = 1$.

Next we assume that the inductive hypothesis holds up to $i - 1$, and we consider J'_i and H'_i . Let us assume that $J(i - 1) = \{1, \dots, m\}$, $c'_1 \geq \dots \geq c'_m$, $H(i - 1) = \{h_1, \dots, h_m\}$, $c'_{h_1} \geq \dots \geq c'_{h_m}$. For the jobs in J'_i and H'_i , we assume that they are numbered so that $J'_i = \{m + 1, \dots, q\}$, $H'_i = \{h_{m+1}, \dots, h_q\}$, so that $c'_{m+1} \geq \dots \geq c'_q$ and $c'_{h_{m+1}} \geq \dots \geq c'_{h_q}$.

We construct a directed graph as follows: the nodes of the graph represent the unit jobs $j \in J(i-1) \cup J'_i \cup H(i-1) \cup H'_i$. (Each job is represented exactly once as a node of the digraph.) For any $j \in J(i-1) \cup J'_i$, we put an edge from job j to job h_j . The graph consists of directed, nontrivial paths and cycles, and isolated nodes with loop edges (trivial cycles), which correspond to jobs j with $j = h_j$. Other than loop-edges, all edges must be of one of the following types, which we label *type-x*, *type-y*, and *type-z*.

Type-x edges (j, k) have $j \in J(i-1)$, $k \in H(i-1)$, $j \neq k$, and, by the inductive hypothesis, $c'_j > c'_k$.

Type-y edges (j, k) have $j \in J'_i$, $k \in H'_i$, $j \neq k$, and $c'_j < c'_k$.

Type-z edges (j, k) have $j \in J'_i$, $k \in H'_i$, $j \neq k$, and $c'_j > c'_k$.

We observe that the final, or sink, nodes t of all paths are precisely those jobs that are in $(H(i-1) \cup H'_i) \setminus (J(i-1) \cup J'_i)$. Similarly, the first, or source nodes s of all paths are those jobs in $(J(i-1) \cup J'_i) \setminus (H(i-1) \cup H'_i)$. In particular, for any sink node t , since t was available when each job j of J'_i was chosen, and since jobs are chosen greedily, we have

$$(1) \quad c'_j > c'_t, \quad \forall j \in J'_i.$$

We claim that for any path's source and sink nodes s and t , $c'_s \geq c'_t$. If the path contains no nodes from J'_i , then every edge is a type-x edge; in that case, by the inductive hypothesis, the values of the nodes monotonically decrease along the path and $c'_s > c'_t$. If the path contains a node from J'_i , consider the first such node u reached from node s along the path (see Figure 5). Then $c'_u > c'_t$, by (1); and since all of the edges preceding u are type-x edges, $c'_s > c'_u$. Thus once again, $c'_s > c'_t$.

Finally, we observe that the only jobs that differ between $J(i-1) \cup J'_i$ and $H(i-1) \cup H'_i$ are the sources and sinks of the paths; all other nodes appear in both sets. We just showed that the source nodes and sink nodes can be paired up so that a job from $J(i-1) \cup J'_i$ always dominates a job from $H(i-1) \cup H'_i$. If we identify each job $j \in (J(i-1) \cup J'_i) \cap (H(i-1) \cup H'_i)$ with itself, and each source node with the sink node on the same path, we have a one-to-one correspondence between $J(i-1) \cup J'_i$ and $H(i-1) \cup H'_i$ such that the former set dominates the latter element-wise. Thus when the sets $J(i-1) \cup J'_i$ and $H(i-1) \cup H'_i$ are each re-ranked as single lists $J(i)$ and $H(i)$, we still have $c'_j \geq c'_{h_j}$, for all j ; and the inductive hypothesis holds for i . $\square$

PROOF OF LEMMA 7. Now we are ready to prove Lemma 7: in Step 4, by the time that job j is compared with its corresponding job h_j at step (*), we have

$$(2) \quad b'_j \geq c'_{h_j}.$$

We will prove something slightly stronger: we will show that, the first time that any job of J'_i is considered in Step 4, for all jobs $j \in J'_i$, $b'_j \geq c'_{h_j}$.

First we consider the case $i = 1$. Because of the greedy choice of small jobs in Algorithm 1 and in moving back unit jobs in Step 3, clearly $b'_j \geq c'_{h_j}$, at the start of Step 4, for all $j \in J'_1$. (Notice that, for this case, $b'_j = c'_j$ for all j .) Thus the lemma is true for J'_1 .

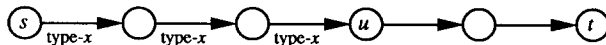


FIGURE 5. A path in the directed graph. Node u corresponds to the first node (tracing from s) that corresponds to a job in J'_i , rather than $J(i-1)$.

Before we prove that (2) holds in general, we give some intuition based on the case $i = 2$. Proving that (2) holds for $i = 2$ is equivalent to showing that the following inequality holds at the beginning of Step 4:

$$(3) \quad \sum_{j \in J'_1} (c'_j - c'_{h_j}) \geq \sum_{j \in J'_2} (c'_{h_j} - c'_j)^+,$$

that is, we require enough residual M_2 -time from J'_1 to cover all of the deficits $(c'_{h_j} - c'_j)$ among jobs in J'_2 . More generally, as we reallocate the M_2 -time to progressively later interval sets, the amount of M_2 -time that we have available for reallocating depends on the aggregate surplus that we have collected over all of the previous interval sets. In other words, the generalization of (3) to the requirement for the i th interval is

$$(4) \quad \sum_{l=1}^{i-1} \sum_{j \in J'_l} (c'_j - c'_{h_j}) \geq \sum_{j \in J'_i} (c'_{h_j} - c'_j)^+.$$

Let us recall the definitions of $J(i-1)$ and $H(i-1)$, from Lemma 10, and let us renumber the jobs, as before, so that $J(i-1) = \{1, \dots, m\}$, $c'_1 > \dots > c'_m$, and $H(i-1) = \{h_1, \dots, h_m\}$, $c'_{h_1} > \dots > c'_{h_m}$. Also, as before, we assume that J'_i and H'_i are numbered $\{m+1, \dots, q\}$ and $\{h_{m+1}, \dots, h_q\}$, respectively, in order of decreasing c'_j and c'_{h_j} . Then (4) is equivalent to

$$(5) \quad \sum_{j \in J(i-1)} (c'_j - c'_{h_j}) \geq \sum_{j \in J'_i} (c'_{h_j} - c'_j)^+,$$

since all that we have done is to rename objects on the left-hand side and pair them up differently. Thus to prove the lemma, it remains to show that (5) holds, for all $i \geq 2$.

We require one more simple observation. Because of the indexing of the sets J'_i and H'_i , for any jobs $j, k \in J'_i$ we have

$$(6) \quad c'_j < c'_k \Leftrightarrow c'_{h_j} < c'_{h_k}.$$

Consider once again the graph constructed in Lemma 10, except that now we delete all of the type- z edges from the graph, i.e., those edges (j, k) with $j \in J'_i$, $c'_j > c'_k$. The remaining graph again consists of paths and cycles. We claim that, for this graph as well, for any (nontrivial) path with source and sink nodes s and t , $c'_s > c'_t$. Furthermore, this claim suffices to prove our original inductive hypothesis. To see this, suppose that the claim is true. Then for any path $(s, j_1, \dots, j_p, t)$ in the new graph, we have

$$0 \leq c'_s - c'_t = (c'_s - c'_{j_1}) + (c'_{j_1} - c'_{j_2}) + \dots + (c'_{j_p} - c'_t).$$

Now let $\mathcal{A}$ be the set of all edges in the digraph, and recall that all edges are of the

form $(j, k) = (j, h_j)$. Since edge-differences around cycles sum to zero, we have

$$\begin{aligned}
 0 &\leq \sum_{\text{all paths}} (c'_s - c'_t) = \sum_{(j,k) \in \mathcal{A}} (c'_j - c'_k) \\
 (7) \quad &= \sum_{\text{type } x} (c'_j - c'_k) - \sum_{\text{type } y} (c'_k - c'_j) \\
 &= \sum_{j \in J(i-1)} (c'_j - c'_{h_j}) - \sum_{j \in J'_i} (c'_{h_j} - c'_j)^+.
 \end{aligned}$$

We observe that this inequality is the same as (5).

Thus to complete the proof of the lemma, we must show that for the new graph (i.e., with the type- z edges deleted), any path with source node s and sink node t satisfies $c'_s \geq c'_t$ (and for $s \neq t$, $c'_s > c'_t$). Note that t cannot be in $J(i-1)$.

Case 1. $t \in (H(i-1) \cup H'_i) \setminus (J(i-1) \cup J'_i)$. Then t was a sink node in the graph before the type- z edges were deleted, and so by the reasoning in Lemma 10, $c'_s > c'_t$.

Case 2. $t \in J'_i$. Then t is one of the new sink nodes created by the deletion of some type- z edge, and so $c'_t > c'_{h_t}$. Without loss of generality, we may assume that $s \in J'_i$, since if $s \in J(i-1)$, we may choose instead to let s be the first node in the path from J'_i . (As we argued before, this node is preceded only by type- x edges, and thus all nodes preceding it have higher value. Also, this new s is well defined, since $t \in J'_i$.) If $s = t$, then clearly $c'_s \geq c'_t$.

Now suppose $s \neq t$. We will show that $c'_s > c'_t$ by induction; specifically, given some node $u \in J'_i$, suppose that, for all nodes $w \in J'_i$ that succeed node u on the path, $c'_w \geq c'_t$; then we claim that $c'_u \geq c'_t$ as well, and if $u \neq t$, $c'_u > c'_t$. The base case, $u = t$, is trivial. Now let $u \in J'_i$, $u \neq t$, such that for all $w \in J'_i$ succeeding u , $c'_w \geq c'_t$. Let w be the first node from J'_i after node u on the path (see Figure 6). Since only type- x edges appear on the path between node h_u and w , $c'_{h_u} \geq c'_w$. By the induction hypothesis, $c'_w \geq c'_t$, and thus

$$c'_{h_u} \geq c'_w \geq c'_t > c'_{h_t}$$

($c'_t > c'_{h_t}$ because edge (t, h_t) was a type- z edge). Thus by (6), $c'_{h_u} > c'_{h_t}$ implies $c'_u > c'_t$. Thus every node $u \in J'_i$ in the path satisfies $c'_u \geq c'_t$, (with $c'_u > c'_t$, for $u \neq t$); and in particular, $c'_s > c'_t$. Thus Case 2 is proved, and the lemma is proved as well. $\square$

PROOF OF LEMMA 8. By Lemma 7, whenever a unit job j is under consideration, $j \in J'_i$, every job k assigned to interval sets $1, \dots, i$ satisfies $b'_k \geq c'_{h_k}$. That fact, along with

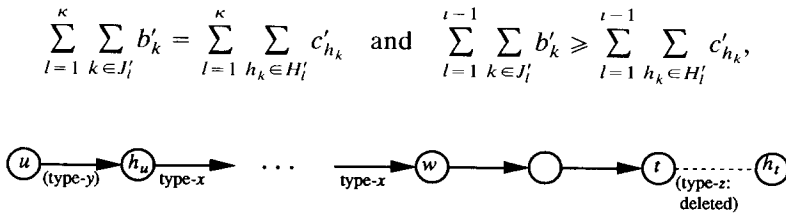


FIGURE 6. A path for Case 2.

implies

$$\begin{aligned}
 \alpha = b'_j - c'_{h_j} &\leq \sum_{k \in J'_i} (b'_k - c'_{h_k}) \\
 &\leq \sum_{l=i+1}^{\kappa} \sum_{h_k \in H'_l} c'_{h_k} - \sum_{l=i+1}^{\kappa} \sum_{k \in J'_l} b'_k \\
 &\leq \sum_{l=i+1}^{\kappa} \sum_{k \in J'_l} (c'_{h_k} - b'_k)^+,
 \end{aligned}$$

so that job j can rid itself of the excess α . $\square$

5. A polynomial approximation scheme. We now describe how the outline scheme yields a polynomial approximation scheme.

THEOREM 2. *There is a polynomial approximation scheme for $F2|r_j|C_{\max}$ based on Algorithm 1 that, for any $\epsilon > 0$, delivers a schedule of length less than $(1 + \epsilon)z^*$ and requires running time*

$$O((1 + 2/\epsilon)^{(12+6\epsilon)/\epsilon^2})(1 + (12 + 6\epsilon)/\epsilon^2)^{1+2/\epsilon} n \log n).$$

PROOF. Fix $\epsilon > 0$, and let $\sum_{i=1}^n a_j = P$. By rounding down the values r_j to the closest multiple of $(\epsilon/2)\max_j r_j$, we have at most $\kappa = 1 + 2/\epsilon$ distinct release dates. Setting $\delta = (\epsilon/6\kappa)P = (\epsilon^2/(12 + 6\epsilon))P$, we have at most $(12 + 6\epsilon)/\epsilon^2$ large jobs and at most $1 + (12 + 6\epsilon)/\epsilon^2$ distinct values for ν_i , $i = 1, \dots, \kappa$. Then by running Algorithm 1 for every possible outline (I, ν) (see Theorem 1), we obtain a $(1 + (\epsilon/2))$ -approximation algorithm for the restricted problem with κ distinct release dates, and (by Lemma 1) a $(1 + \epsilon)$ -approximation algorithm for $F2|r_j|C_{\max}$. The number of possible outlines is bounded by the number of ways of assigning effective release dates to large jobs times the number of ways of choosing the ν_i , or

$$(1 + 2/\epsilon)^{(12+6\epsilon)/\epsilon^2} \cdot (1 + (12 + 6\epsilon)/\epsilon^2)^{1+2/\epsilon},$$

which is a fixed number (in terms of ϵ). Algorithm 1 requires $O(n \log n)$ time, so that the overall running time is

$$O((1 + 2/\epsilon)^{(12+6\epsilon)/\epsilon^2} \cdot (1 + (12 + 6\epsilon)/\epsilon^2)^{1+2/\epsilon} n \log n). \quad \square$$

This running time emphasizes the fact that our algorithm is mainly of theoretical rather than practical interest, because of the huge constant involving $1/\epsilon$. Indeed, for $\epsilon = 1/2$ this constant is already $5^{60} \cdot 61^5$.

In conclusion, we are optimistic that outline schemes may lead to polynomial approximation schemes for other constrained scheduling problems. In particular, techniques similar to those presented here might work for the more constrained problem $F2|r_j|L_{\max}$. However, the analysis for this case is complicated by the fact that we can no longer assume that there is a permutation schedule that is optimal.

Acknowledgements. I am greatly indebted to David Shmoys for the countless discussions on scheduling theory that we have had, and I thank him for pointing out a more elegant proof to Lemma 2 than I had originally included. I would also like to

thank the referees for their careful reading of this paper and their insightful suggestions for improving it.

References

- Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Co., NY.
- Graham, R. L. (1966). Bounds for Certain Multiprocessing Anomalies. *Bell System Technical J.*, **45** 1563–1581.
- _____, Lawler, E. L., Lenstra, J. K. and Rinnooy Kan, A. H. G. (1979). Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey. *Ann. Discrete Math.* **5** 287–326.
- Hall, L. A. (1989). *Two Topics in Discrete Optimization: Polyhedral Structure of Capacitated Trees and Approximation Algorithms for Scheduling*. Ph.D. Thesis, MIT, Cambridge, MA.
- _____, and Shmoys, D. B. (1989). Approximation Algorithms for Constrained Scheduling Problems. In *Proc. IEEE 30th Annual Symp. Foundations of Computer Science*, pp. 134–139.
- _____, and _____ (1990). Near-Optimal Sequencing with Precedence Constraints. In *Proc. Math. Prog. Soc. Conference on Integer Programming and Combinatorial Optimization*, University of Waterloo, pp. 249–260.
- _____, and _____ (1992). Jackson's Rule for Single-Machine Scheduling: Making a Good Heuristic Better. *Math. Oper. Res.* **17** 22–35.
- Johnson, S. M. (1954). Optimal Two- and Three-Stage Production Schedules with Setup Times Included. *Naval Res. Logist. Quart.* **1** 61–68.
- Kovaliev, M. Y. (1985). An Efficient ϵ -approximation Algorithm for Maximizing Completion Time in Two-Stage Systems of Parallel Machines. *Isv. Akad. Nauk, B.S.S.R., Phys.-Math. Nauk Series* **3** 118. (Russian).
- Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G. and Shmoys, D. B. (1989). Sequencing and Scheduling: Algorithms and Complexity. In *Handbooks in Operations Research and Management Science, Volume 4: Logistics of Production and Inventory*, (S. C. Graves, A. H. G. Rinnooy Kan, and P. Zipkin, eds.), North-Holland.
- Lenstra, J. K., Rinnooy Kan, A. H. G. and Brucker, P. (1977). Complexity of Machine Scheduling Problems. *Ann. Discrete Math.* **1** 343–362.
- Potts, C. N. (1985). Analysis of Heuristics for Two-Machine Flow-Shop Sequencing Subject to Release Dates. *Math. Oper. Res.* **10** 576–584.

L. A. Hall: Department of Mathematical Sciences, Johns Hopkins University, Baltimore, Maryland 21218

Copyright 1994, by INFORMS, all rights reserved. Copyright of Mathematics of Operations Research is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.